

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа

“ЛР3: Контейнеризация написанного приложения
средствами docker”

Выполнил:

Преображенский Артемий

Группа:

К3341

Проверил:

Добряков Д. И.

Санкт-Петербург

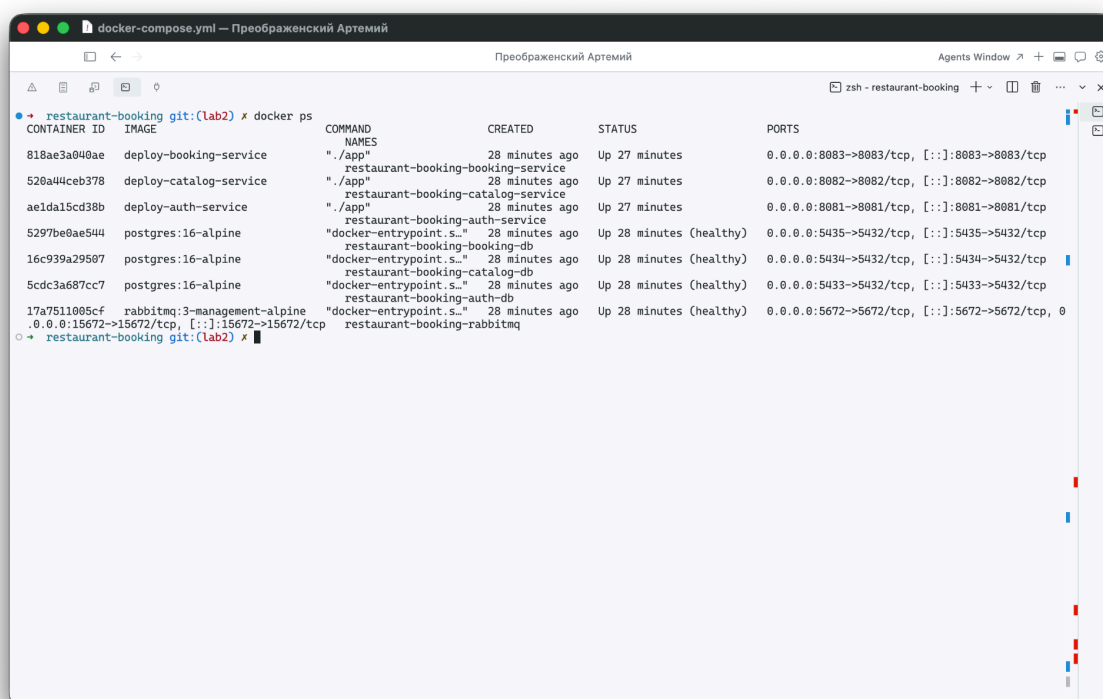
2026 г.

Задача

1. реализовать Dockerfile для каждого сервиса;
2. написать общий docker-compose.yml;
3. настроить сетевое взаимодействие между сервисами.

Ход работы

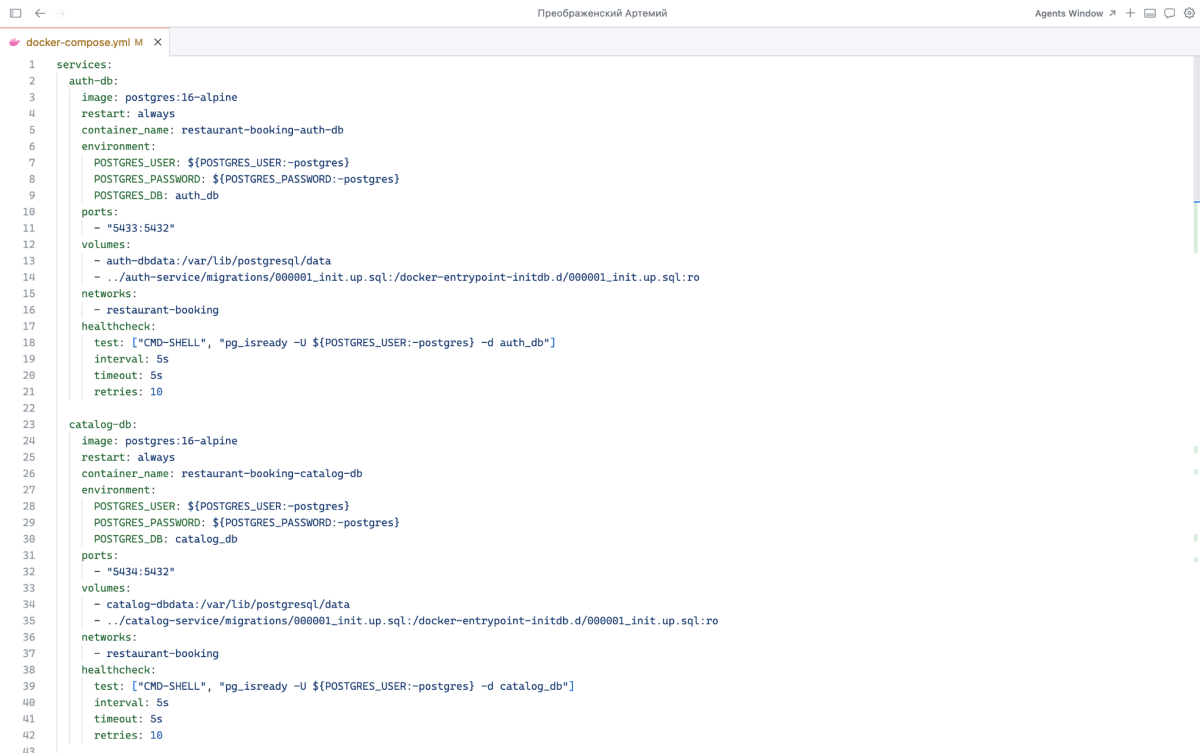
Для трёх приложений (auth, catalog, booking) использован один общий deploy/Dockerfile с аргументом сборки SERVICE: при docker compose build для каждого сервиса подставляется своё имя, собирается соответствующий бинарник из cmd, в образ на Alpine кладётся только он.



В deploy/docker-compose.yml собрано всё окружение: три контейнера PostgreSQL (отдельная БД под каждый сервис), RabbitMQ, три микросервиса. У БД и брокера настроены healthcheck, у приложений — depends_on, чтобы они стартовали после готовности баз и очереди. Миграции подключаются через docker-entrypoint-initdb.d.

Все сервисы в одной пользовательской сети restaurant-bookings. В переменных окружения вместо localhost указаны имена сервисов из compose (auth-db, catalog-db, booking-db, auth-service, catalog-service, rabbitmq и т.д.), поэтому HTTP и DSN работают между контейнерами без

дополнительной настройки. С хоста доступ по сброшенным портам (8081–8083, БД, при необходимости 15672 для UI RabbitMQ).



```
1 services:
2   auth-db:
3     image: postgres:16-alpine
4     restart: always
5     container_name: restaurant-bookings-auth-db
6     environment:
7       POSTGRES_USER: ${POSTGRES_USER:-postgres}
8       POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-postgres}
9       POSTGRES_DB: auth_db
10    ports:
11      - "5433:5432"
12    volumes:
13      - auth-dbdata:/var/lib/postgresql/data
14      - ../auth-service/migrations/000001_init.up.sql:/docker-entrypoint-initdb.d/000001_init.up.sql:ro
15    networks:
16      - restaurant-bookings
17    healthcheck:
18      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER:-postgres} -d auth_db"]
19      interval: 5s
20      timeout: 5s
21      retries: 10
22
23   catalog-db:
24     image: postgres:16-alpine
25     restart: always
26     container_name: restaurant-bookings-catalog-db
27     environment:
28       POSTGRES_USER: ${POSTGRES_USER:-postgres}
29       POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-postgres}
30       POSTGRES_DB: catalog_db
31    ports:
32      - "5434:5432"
33    volumes:
34      - catalog-dbdata:/var/lib/postgresql/data
35      - ../catalog-service/migrations/000001_init.up.sql:/docker-entrypoint-initdb.d/000001_init.up.sql:ro
36    networks:
37      - restaurant-bookings
38    healthcheck:
39      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER:-postgres} -d catalog_db"]
40      interval: 5s
41      timeout: 5s
42      retries: 10
43
44 networks:
45   restaurant-bookings:
```

Вывод

В ходе работы реализована контейнеризация: единый параметризованный Dockerfile собирает три микросервиса, общий deploy/docker-compose.yml поднимает приложения вместе с тремя изолированными экземплярами PostgreSQL и RabbitMQ.